There are three basic ways that programmers deal with HTML from inside their own languages: templates, hooks, and language DSLs that allow them to compose HTML entirely inside their program. One way to compose HTML inside CoffeeScript is with a small DSL:

```
loggedIn = -> true

doctype 5
html ->
    body ->
      ul class: 'info', ->
        li -> 'Logged in' if loggedIn()
```

Here's the HTML generated by this CoffeeScript DSL (with newlines and indentation added for readability):

```
<!DOCTYPE html>
<html>
  <body>
    <ul class='info'>
      <li>Logged in</li>
```

The nice thing about this approach to HTML is that you have the full power of CoffeeScript in your HTML. In the next listing you can see a basic implementation of this HTML DSL that supports a subset of HTML elements.

Listing 8.4. A basic DSL for HTML

```
doctype = (variant) ->
  switch variant
    when 5
      "<!DOCTYPE html>"

markup = (wrapper) ->
  (attributes..., descendents) ->
    attributesMarkup = if attributes.length is 1
      ' ' + ("#{name}='#{value}'" for name, value of attributes[0]).join ' '
    else
      ''
    "<#{wrapper}#{attributesMarkup}>#{descendents() || ''}</#{wrapper}>"

html = markup 'html'
body = markup 'body'
ul = markup 'ul'
li = markup 'li'
```

In this example, only a single doctype is supported.

Attributes for the HTML element are contained in all of the parameters except for the last one, which is always reserved for a function containing any nested HTML elements.